COMPILER CONSTRUCTION LAB · WS 2026/27 · PA1 · GROUP 5

Lexing & Parsing ChocoPy: Challenges, Fixes and Open Problems

The engineering log of our front-end: what broke, why it broke, how we found out, and what is still open.

Mithulram Gunasekaran · Mohamed Nuhuman Mohamed Nafeel · Rajapaksha Mudiyanselage Dinodi Divyanjana Rajapaksha

33/33	44/44	3,000+	0	1.37
provided sample tests	our own edge-case tests	random programs, all identical to the reference	LALR conflicts in the grammar	errors per real mistake (reference: 2.01), 0 aborted parses

1. The task in one paragraph
2. How we approached it
3. Challenges in the lexer (C1–C9)
4. Challenges in the parser (C10–C16)
5. Challenges in error recovery (C17–C21)
6. Challenges in tooling and testing (C22–C24)
7. Bugs we found in the reference
8. Open problems and known limitations
9. What we would do differently
10. What we referred to

1. The task in one paragraph

PA1 is the front-end of our ChocoPy compiler. We had to write a JFlex lexer (`ChocoPy.jflex`) and a CUP grammar (`ChocoPy.cup`) that turn a ChocoPy program into an AST in JSON. The grader compares our JSON with the staff reference parser's JSON: for a **valid** program the whole tree must be equal, **source locations included**. For an **invalid** program, only the line and column of the **first** syntax error are compared. The lexer may never report errors itself: it emits an `UNRECOGNIZED` token, and the parser reports `Parse error near token X: text` . The parser also has to *recover* from errors and keep parsing.

THE REAL DIFFICULTY

Writing a parser that *accepts* ChocoPy is a textbook exercise. Producing a tree that is **identical to the reference down to the last column number** was the hard part. The reference has location conventions (and a few quirks) that no document describes. Most of our work went into finding them out and copying them exactly.

2. How we approached it

We treated the reference implementation as an **oracle** and worked from evidence rather than assumptions. The work went through five phases:

1 · Read
Spec, manual §3–4 (lexical rules, grammar, precedence), presentation guidelines.

›

2 · Probe
Ran tiny probe files through the *reference* lexer and parser to learn token names and location rules.

›

3 · Build
Lexer (indentation first), then the grammar from Figure 3, conflict-free.

›

4 · Fuzz
Random program generator + differential tester using the grader's own comparison rule.

›

5 · Measure
A metric for error-recovery quality; kept only the changes that measurably improved it.

Two examples of the probing in phase 2:

```
$ java -cp "chocopy-ref.jar:target/assignment.jar" chocopy.reference.ChocoPyLexer ops.py
--+--PLUS-- --//--IDIV-- --==--EQEQ-- --=--EQ-- --[--LINDEX-- --async--RESERVED-- --"\o"--
UNRECOGNIZED-- ...

$ java ... chocopy.ChocoPy --pass=r coverage.py      # where does each node start and end?
IfStmt [16,13,19,8]     <- ends at column 8 of line 19: the indentation of the NEXT line
```

That second line was the key to the location rules described in C4.

3. Challenges in the lexer

1 One line can close many blocks, but an action returns one token solved

SYMPTOM A line that dedents from level 12 to level 0 needs three DEDENT tokens, but a JFlex action can only `return` one symbol.

OPTIONS (a) A queue of pending tokens, with the generated `next_token()` wrapped. (b) Re-scan the same whitespace once per DEDENT.

FIX We chose (b). `processIndentation()` pops one level, builds one DEDENT, then calls `yypushback(yylength())`. The next call matches the same whitespace again, compares it with the smaller stack, and emits the next DEDENT, until the levels agree. There is no extra data structure: the indentation stack is the only state.

WHERE `ChocoPy.jflex` lines 175–220, and the line-start rule at 309–315.

LESSON Changing the *input* was simpler than adding a token buffer, and it can't fall out of sync with JFlex's line/column counters.

2 Indentation zero is an *empty* match solved

SYMPTOM A line at column 1 has no leading whitespace, so a rule like `{WhiteSpace}*` would match the empty string. JFlex cannot make progress on an empty match, so it would never fire.

FIX The rule matches the whitespace **plus the first real character**, `{WhiteSpace}* [^ \t\r\n#]`, and gives that character back at once with `yypushback(1)`. So the match is never empty, and `yytext()` is exactly the indentation.

3 Blank lines and comment-only lines solved

SYMPTOM The manual (§3.1.4) says a line with only spaces, tabs and/or a comment produces **no** NEWLINE, and its indentation does not count. At first a comment indented deeper than its block produced a bogus INDENT.

FIX Three ignore-rules in `YYINITIAL`: `ws* comment? linebreak`, and two variants for the last line of a file with no line break (`ws* comment` and `ws+`). JFlex's *longest match* makes sure that "`  x`" goes to the indentation rule, while "`  # c\n`" goes to the blank-line rule.

4 Where exactly is a DEDENT? (the location puzzle) solved

SYMPTOM Our first ASTs differed from `coverage.py.ast` only in the end column of `IfStmt`, `WhileStmt` and `ForStmt`.

FOUND BY Reading the expected output: `IfStmt [16,13,19,8]` ends at line 19, column 8, and line 19 is indented by 8 spaces. A compound statement ends at the DEDENT that closes it, and that **DEDENT spans the leading whitespace of the next line**: `[L,1,L,width]`. With no indentation the span is `[L,1,L,0]`, which matches the sample error `DEDENT [5,1,5,0]` in `bad_func_def.py.ast`.

FIX Build the DEDENT symbol *before* pushing the whitespace back. Otherwise `yylength()` is already 0.

5 End of file: missing newline, and DEDENTs whose columns keep growing mirrored a quirk

SYMPTOM (a) A file whose last line has no `\n`. (b) Our random tester found programs where the reference's `Program` ended at `[12,2]` and ours at `[12,1]`.

FOUND BY The reference lexer prints the three DEDENTs at end of file as `(5,1) (5,2) (5,3)`: each one sits one column further right. So nested blocks that all end at EOF get different end columns. For a missing final newline, the reference behaves as if a `\n` had been appended: the DEDENTs land on the *next* line, column 1.

FIX The `<<EOF>>` rule first emits the missing NEWLINE (if we are still in state `CODE`), then one DEDENT per call, then EOF. `eofSymbol()` moves one column right for every token it makes up.

NOTE We reported this as a reference bug (Section 7): those columns don't exist in the source. We still copy it, because the grader compares locations.

6 A dedent to a level that was never opened solved, better than reference

SYMPTOM Levels 0, 4, 8 and then a line at 2. Python says "unindent does not match any outer indentation level", but our lexer must not report errors.

FIX Pop every level larger than 2, emitting one DEDENT each (flag `dedenting`). If the stack then sits *below* the line's width, emit an `UNRECOGNIZED` token, which no grammar rule accepts, and treat the line as part of the enclosing level we reached.

V1 → V2 Our first version, like the reference, opened a new level with an INDENT after the error. The parser's recovery threw that INDENT away, but its matching DEDENT turned up one line later as a second, misleading error. Not opening a level removes the cascade.

BONUS The reference pops only *one* level here, ends up with more INDENTs than DEDENTs, and reports a bogus second error at EOF. That is our bug report #1.

7 String literals: a build error, a redesign, and two quirks solved

V1 (BUILD ERROR) We wrote the allowed characters as `[\u0020\u0021\u0023-\u005B\u005D-\u007E]`, and JFlex failed with "Syntax error". JFlex replaces `\uXXXX` escapes **before** it parses the regex (as `javac` does), so `\u005B` turned into a bare `[` that broke the character class. Fix: write it as `[ !#-\[\]-~]`.

V2 (REDESIGN) Our first version scanned strings character by character in a separate `STRING` state. The differential tester showed that the reference reports an unterminated string as a **single** `"` token and keeps lexing after it. We replaced the whole state with one regex for a complete literal plus a Java decoder (`stringSymbol()`, lines 239–268). That was simpler and it matched the reference.

QUIRK 1 The reference's string match runs **across line breaks** up to the next quote, and then rejects the literal (a newline is not printable ASCII). Its end location is computed as start + length *on the start line*. We copied this.

QUIRK 2 A backslash directly before a line break ends the match (`\\[^\r\n]`). The fuzzer found this one too.

RULES Only `\" \n \t \\` are legal. Any other escape (`"C:\path"`) or a character outside 32–126 (such as a tab) makes the **whole literal** one `UNRECOGNIZED` token. A string whose contents look like an identifier is an `IDSTRING` (allowed as a type, e.g. `self: "Node"`).

8 Integer literals: overflow, leading zeros, and negative numbers solved

BEHAVIOUR `2147483647` is the largest literal. `2147483648` becomes `UNRECOGNIZED` (caught as a `NumberFormatException`), like the reference. `007` lexes as `0 0 7` (longest match with `0 | [1-9][0-9]*`), so the parser fails at the second `0`.

SUBTLE So `-2147483648` cannot be written as a literal at all: unary minus is applied to an out-of-range literal. The manual's rule implies this, and the reference agrees, so we left it as it is.

9 Python keywords that ChocoPy reserves solved

BEHAVIOUR `lambda`, `import`, `async`, … (16 words) may not be identifiers (manual §3.3), but no rule uses them. We emit a `RESERVED` token (the reference's name for it), so `lambda = 5` gives "Parse error near token RESERVED". Because keywords come before the identifier rule and JFlex prefers the longest match, `classic` or `iffy` still lex as identifiers.

4. Challenges in the parser

10 The text in error messages was wrong solved

SYMPTOM

The starter's `syntax_error` printed `lexer.yytext()`. That is the text of the *most recent match*, which is not always the offending token's text: after `yypushback` (DEDENT), for tokens we invent at EOF, and while CUP reads ahead during recovery.

FIX

The lexer stores each token's source text in an `IdentityHashMap<Symbol,String>`, and the parser asks `getTokenText(cur_token)`. The text is attached to the token itself, so no lexer state is involved.

11 Precedence without a tower of nonterminals, and zero conflicts solved

APPROACH

We kept the manual's `expr` / `cexpr` split and resolved everything else with CUP `precedence` lines (lowest → highest: `if-else`, `or`, `and`, `not`, comparisons, `+ -`, `* // %`, unary `-`, `. []`).

DETAILS

- Comparisons are `nonassoc`, so `a < b < c` is an error, as in the manual.
- `not` / `and` / `or` / `if` only work on `expr`, so `True == not False` is a syntax error, exactly as the manual demands.
- Unary minus gets its own level with a dummy terminal and `%prec UMINUS`, so `-a * b` is `(-a) * b`, but `-a.b` is `-(a.b)`.
- `x if c else y if d else z` is right-associative.

RESULT

The grammar builds with **zero** shift/reduce and reduce/reduce conflicts. Our `pom.xml` has no `expect` option, so even a single conflict would fail the build.

12 Declarations must come before statements: `x : int` vs `x = 1` solved

QUESTION

Both a variable definition and a statement can start with an identifier. Is the grammar still LALR(1)?

ANSWER

Yes. After shifting `IDENTIFIER`, the lookahead decides: `:` means a typed variable, and anything else means an expression or assignment target. The same goes for assignment: after an identifier or member/index expression, `=` reduces it to a `target`, which is how `a = b.c = d[0] = 1` collects all three targets.

13 Locations from symbols, not from AST nodes solved: found by the fuzzer

SYMPTOM Two random programs failed: one that began with `pass`, and one with `else: pass`.

ROOT CAUSE `pass` builds **no AST node**. We computed the program's start, and an `if`'s end, from the AST nodes and from whether lists were empty. The reference uses **grammar-symbol** positions: a program starting with `pass` starts at that token, and an `if` with `else: pass` still ends at the else-block's DEDENT, although its else-list is empty.

FIX We split the rules (`IF … block` vs `IF … block elif_else`; `program ::= top_decls stmt_list | top_decls`) so each location comes from a symbol (`restxright`, `sxleft`). We never test whether a list is empty to decide a location.

14 Desugaring `elif` solved

REQUIREMENT The AST has no elif. `if a: X elif b: Y else: Z` must become `IfStmt(a, X, [IfStmt(b, Y, Z)])`.

FIX A right-recursive `elif_else` rule: each `elif` returns a one-element list holding a nested `IfStmt`, whose location runs from `elif` to the end of the rest of the chain. No post-processing is needed. (`ChocoPy.cup` lines 575–588)

15 A function without `-> type` solved

BEHAVIOUR The AST still needs a return type: the reference uses `ClassType "<None>"` **located at the header's colon** (we found this with a probe). We capture the colon as `COLON:c` and build that node when `return_type_opt` is empty.

16 Why does a class end at `[36,0]` but its method at `[34,22]`? solved

PUZZLE In `coverage.py.ast` the method `baz` ends at its last NEWLINE (34,22), yet the class around it ends at (36,0), the DEDENT of line 36.

EXPLANATION A `FuncDef` **node** ends where its body ends, but the `func_def` grammar **symbol** also includes the closing DEDENT. The class takes its end from its last member's *symbol*, so it ends at that DEDENT. Seeing that "node location" and "symbol location" are two different things made all the location rules fall into place.

5. Challenges in error recovery

17 A reduce/reduce conflict hiding in error recovery solved

PROBLEM

The natural design is to have every list absorb errors: declarations (`decls ::= decls error`) and statements (`stmts ::= error`). But right after the declarations, a declaration *or* a statement can begin. If both `error` rules are live in that state, shifting `error` leads to a state with two completed items: a **reduce/reduce conflict**.

FIX

Separate nonterminals with separate roles. The top-level `stmt_list` only accepts `error` *after* its first statement, while `block_stmts` (inside an indented block or a function body, where nothing competes) may also start with `error`. An error on the very first top-level statement is absorbed by `top_decls`, which is harmless.

18 One mistake, five errors: cascades improved and measured

SYMPTOM

`x:int = 1 + 2` reported errors on *this and every later declaration*, and so did the reference. Recovery resumed on the leftover `2` as a **statement**, and since declarations may not follow statements, every following `def` / `class` failed.

FIX

Anchored recovery rules. A bad initial value skips to the end of its line (`typed_var EQ error NEWLINE`). A broken `def` / `class` / `if` / `elif` / `while` / `for` header skips to the end of *its* line, and if an indented body follows, the body is parsed too, so its INDENT doesn't cause more errors (`IF error NEWLINE INDENT block_stmts DEDENT`).

SAFETY

Adding `error` rules cannot move the **first** error. `error` never appears in real input, so the set of valid token prefixes is unchanged, and the first token an LR parser rejects stays the same. The differential tester confirmed this on every program.

DEMO

`demo/4_recovery.py` (3 real mistakes): ours reports **3 errors**, and the reference reports **8**.

19 The vanishing statement

solved: Bison-style default reductions

SYMPTOM

In `z = 5` followed by `lambda = 5`, the *correct* statement `z = 5` was missing from the AST, in both our parser and the reference.

ROOT CAUSE

To reduce `stmt ::= simple_stmt NEWLINE`, the LR parser needs a *valid* lookahead, and RESERVED is not one. So it reported the error *before* the finished statement was reduced, and recovery popped `z = 5` off the stack.

FIX

Default reductions in **consistent states** (states whose only action is one reduction), as Bison does by default. CUP has this as `-compact_red`, but that flag is set in `pom.xml`, which we may not edit. So `enableDefaultReductions()` patches CUP's action table when the parser starts (`init with`). The error position cannot change: those reductions would have happened on every valid token anyway, and reductions consume no input.

SIDE EFFECT

It exposed C20: once a block is fully reduced, a later error reached a recovery rule that was meant for the header.

20 The abort trap: anchors that never come

solved: found by a better metric

SYMPTOM

After C19, one test file suddenly produced an **empty AST**: CUP had given up on the whole file. When we added an "aborted parses" column to our metric, we saw that the earlier version had also aborted on 4 of 133 programs (the reference: 1–2). The old metric had hidden this.

ROOT CAUSE

Our first anchored rules waited for `COLON NEWLINE INDENT` (`IF error COLON block`). CUP discards tokens until the anchor parses. If it never comes (a missing colon, or an error *after* the block that fell back into the "after IF" state), CUP reaches EOF and **gives up**. A second case: a function whose *only* statement is broken. With `func_decls error`, recovery resumed in declaration mode, threw away the function's DEDENT, and failed at EOF. The reference does exactly this too.

FIX

(1) Anchor only on **NEWLINE**, which always comes soon, and make the indented body optional. (2) Move the normal headers into small nonterminals (`if_header ::= IF expr COLON` …): once reduced, their internal states leave the stack, so later errors can't fall back into them. (3) Function bodies use `block_stmts`, which may start with `error`. `func_decls error` is gone, since two live error rules would be a reduce/reduce conflict.

ALSO

Our first abort detector ("the AST is empty") gave false positives: a program that is one broken `for` loop correctly yields an empty AST. The tool now checks CUP's own "*Error recovery fails*" message.

21 Tuning CUP's re-synchronisation window `decided with data`

BACKGROUND After an error, CUP trusts its recovery only if the next `error_sync_size()` tokens (default 3) parse cleanly. Otherwise it discards a token and tries again. That is why `lambda = 5` right after another broken line goes unreported: it falls inside the window.

EXPERIMENT We overrode it to 2 and 1. Average errors went 2.86 → 2.93 → 3.47, so smaller windows give *more* cascades. We kept the default of 3.

The measurements behind section 5

`tools/recovery_quality.py` puts exactly **one** `$` into one random line of each valid program. Ideal: 1 error, all undamaged code kept, never give up. Two independent corpora, final version:

Corpus	Parser	Exactly one error	Avg. errors per mistake	Undamaged code kept	Aborted parses
A (133 programs)	Reference	43.6 %	2.28	95.6 %	1
	Ours	**76.7 %**	**1.35**	**95.8 %**	**0**
B (213 programs)	Reference	46.0 %	1.84	96.1 %	2
	Ours	**73.2 %**	**1.38**	**96.2 %**	**0**

How we got there, on corpus A: list-level `error` rules only → 2.86 errors per mistake. Colon-anchored rules → 1.30, but 4 aborted parses. Final design (NEWLINE anchors, header nonterminals, default reductions) → 1.35 with 0 aborted parses and more code kept. We accepted slightly more errors than the colon version in exchange for **never** giving up on a file.

6. Challenges in tooling and testing

22 The very first build failed (and it wasn't our code) `understood`

SYMPTOM `mvn clean package` on the untouched starter failed with HTTP **502 Bad Gateway** for `jackson-bom 2.21.3`.

ROOT CAUSE `pom.xml` asks for `jackson-databind` with the **open version range** `[2.9.10.1,)`, so Maven asks Maven Central for "the newest version" on every build. That is a network round-trip that can fail, and the result can change over time. A retry worked. We may not edit `pom.xml`, so for the live demo we make sure the dependencies are already in the local Maven cache (see the Demo Runbook).

ALSO On JDK 27, Jackson prints "Final field … mutated reflectively" warnings. These are harmless and come from the provided `chocopy.common` code.

23 Passing 33 samples proves little: building a differential tester done

WHY

Grading also uses **private** tests. We passed all 33 samples within hours, and the fuzzer still found 13 differences afterwards.

TOOLS

`tools/fuzz.py` generates random programs from the grammar, with random tabs/spaces, CR/LF/CRLF line endings, comments, blank lines and missing final newlines. It also makes a mutated copy of each with one random defect. `tools/difftest.py` runs both parsers and applies the **grader's rule**: full AST equality for valid programs, first-error location for invalid ones. It runs in parallel threads, because each run starts two JVMs.

FOUND

C5(b), C7 (v2 and both quirks) and C13 were all found this way. None of them showed up in the samples. Final run: **3,511 files** (samples, edge cases, probes, four random corpora), all identical to the reference under the grader's rule.

24 Testing the tester: our generator lied fixed

SYMPTOM

While measuring recovery, we saw errors *before* the injected defect. Only **193 of 400** "valid" generated programs were actually valid.

ROOT CAUSE

The generator built assignment targets such as `-x.foo = 1` (which parses as `-(x.foo)`, not assignable) and comparisons nested inside arithmetic (`a < b + c is d`, a chained comparison).

FIX

Targets are now built only from assignable expressions, and every comparison is parenthesised. After that, **500/500** programs were valid, and all **1,000** (valid + mutated) matched the reference. Both parsers had agreed on the invalid ones from the start, but this way we really do compare 500 ASTs.

7. Bugs we found in the reference implementation

These are also in the README write-up, as the spec asks (§8.5).

Bug	Input → reference behaviour	Expected
1. Unbalanced INDENT/DEDENT after a bad dedent	Levels 0,4,8 then a line at 2: emits `DEDENT, UNRECOGNIZED, INDENT` (pops only one level and pushes 2 on top of 4, so the stack is no longer increasing). Ends with 3 INDENTs and only 2 DEDENTs, and reports a bogus `Parse error near token EOF`.	Pop every level above 2 first, so only the error at the bad line is reported (as our lexer does).
2. Made-up columns for DEDENTs at EOF	Three blocks ending at EOF get DEDENTs at (5,1), (5,2), (5,3), so nested `IfStmt`s end at columns that don't exist. The token text is also stale: `Parse error near token DEDENT: x`.	All at (5,1), with empty text.
3. Strings across lines	An unclosed quote swallows everything up to the next quote, even many lines later, as one token. Its end location is computed on the *start* line: `[18,9,18,14]` for a token that ends on line 19.	Stop at the end of the line (like Python's "unterminated string literal"), or at least report the right end line.

8. Open problems and known limitations

- **We copy two reference quirks on purpose** (EOF DEDENT columns, multi-line string tokens). If the reference were fixed, we would revert `eofColumn += 1` and change one character class. Both places are commented.
- **A statement with a broken header is dropped.** Its body is parsed and checked for errors, but no node is kept. We could build a placeholder node with a missing condition, but that puts half-valid nodes into the tree that PA2 would have to handle, so we chose not to.
- **A broken declaration inside a function body** (other than a bad initial value) switches recovery to statement mode, so a later declaration in the *same* function is reported as well. This follows from C20 (we removed `func_decls error` to prevent aborts).
- **An error right after an error can go unreported** (C21, `lambda = 5`), because of CUP's 3-token window.
- **Error messages:** the wording differs slightly from the reference, which adds hints such as `<bad escape>`. Only locations are graded. A message like "unindent does not match any outer level" would be friendlier than "UNRECOGNIZED".
- The lexer's debug `main` (generated by `%cupdebug`) prints `yytext()`, so a DEDENT shows empty text there. This is cosmetic.

9. What we would do differently

- **Build the differential tester on day one.** Every hard bug was found by it, not by the samples.
- **Probe the reference before designing.** Our STRING state (C7) was written before we knew how the reference treats unterminated strings, and it had to be thrown away.

- **Measure the failure you fear most first.** Our recovery metric counted errors but not aborted parses, so a version that "improved" the error count was quietly giving up on 3% of files (C20).
- **Make locations a first-class design decision:** "symbol position or node position?" should be answered once for every rule, not discovered rule by rule.

10. What we referred to

- PA1 specification, ChocoPy Language Manual and Reference §3 (lexical structure) and §4 (grammar, Figure 3, precedence table).
- JFlex user manual: lexical states, `yypushback`, `<<EOF>>`, longest match and rule order, `\u` pre-processing.
- CUP manual: `precedence`, `%prec`, the `error` symbol, `error_sync_size`, `xleft/xright` with `ComplexSymbolFactory`.
- Python 3 language reference, "Lexical analysis → Indentation" (the source of the manual's algorithm).
- The staff reference implementation, used as a black-box oracle (`--pass=r`, `chocopy.reference.ChocoPyLexer`).